Ex. No: **Chest X-Ray Image Classification using a Convolutional Neural Network (CNN)**

Date:

Problem Statement:

Accurate radiological classification of pulmonary conditions is a fundamental challenge in medical imaging, thoracic medicine, and clinical computer-aided diagnostics. In the case of acute respiratory infections, conditions such as Pneumonia exhibit subtle radiological signatures on chest radiographs—specifically patchy ground-glass opacities, lobar consolidations, and bronchovascular markings—that visually overlap with normal anatomical structures or benign variations. Manual inspection by certified radiologists is labor-intensive, time-consuming, and prone to inter-observer variability, particularly in high-volume emergency departments or resource-constrained clinical centers where access to experienced thoracic specialists is severely limited.

There is, therefore, an urgent need for an automated, fast, and accessible machine learning system that can analyze continuous two-dimensional radiographic pixel data and classify the radiograph into its correct clinical category—specifically NORMAL or PNEUMONIA—with a measurable confidence score. This project designs and implements an end-to-end, web-based Chest X-Ray Classification system using a Convolutional Neural Network (CNN) architecture built with TensorFlow, Keras, and Scikit-Learn. The system encompasses the complete 10-step machine learning lifecycle: data ingestion, automated directory quality verification, exploratory data analysis (EDA), uniform bicubic resizing to 128×128 pixels, 3-channel RGB conversion, min-max pixel intensity normalization [0, 1], data augmentation (rotation, translation, zoom, horizontal flip), stratified train/validation/test partitioning, CNN model compilation and training with EarlyStopping and ModelCheckpoint callbacks, test evaluation on unseen data (Accuracy, Precision, Recall, F1-Score, Confusion Matrix), and real-time inference served through an interactive clinical-style Streamlit dashboard with Explainable AI (Grad-CAM).

Objectives:

1. To collect and inspect the labelled Chest X-Ray radiograph dataset organized into binary classes: NORMAL and PNEUMONIA.

2. To conduct Exploratory Data Analysis (EDA) and visualize class distributions, total sample counts, and comparative anatomical features.

3. To partition the dataset into stratified Training (70%), Validation (15%), and held-out Test (15%) sets using reproducible random sampling.

4. To apply standardized preprocessing: bicubic spatial resizing to 128×128, 3-channel RGB conversion, and min-max pixel intensity scaling [0.0, 1.0].

5. To apply controlled data augmentation (rotation, translation, zoom, horizontal flip) strictly on training data to prevent overfitting without data leakage.

6. To design, compile, and train a 3-block Convolutional Neural Network (CNN) architecture with Conv2D, ReLU, MaxPooling2D, Dropout, and Softmax output.

7. To evaluate the trained model on an unseen held-out test partition, calculating Accuracy, Precision, Recall, F1-Score, and plotting the Confusion Matrix.

8. To implement Explainable AI using Grad-CAM attention heatmaps and build a responsive, clinical-style Streamlit dashboard with synthetic patient vitals.

Software / Hardware Required:

- **Operating System:** Windows 10 / 11 / Linux / macOS
- **IDE:** Visual Studio Code / Antigravity IDE
- **Language:** Python 3.x
- **Deep Learning Framework:** TensorFlow 2.x / Keras (Sequential, Conv2D, MaxPooling2D, Dropout, Dense)
- **Machine Learning:** scikit-learn (train_test_split, accuracy_score, precision_score, recall_score, f1_score, confusion_matrix)
- **Data Processing:** Pandas, NumPy
- **Image Processing:** Pillow (PIL)
- **Data Visualization:** Matplotlib, Seaborn
- **Model Serialization:** Native Keras (.keras format)
- **Frontend Dashboard:** Streamlit (Clinical Analytics UI)
- **Dataset:** Public Chest X-Ray Radiographs (NORMAL and PNEUMONIA)
- **Version Control:** Git

Project Directory Structure:

The project ("Chest_Xray_CNN_Project") is organized as a modular deep learning pipeline with separate dataset, trained-model, generated evaluation plots, synthetic clinical records, and frontend presentation modules, as shown below.

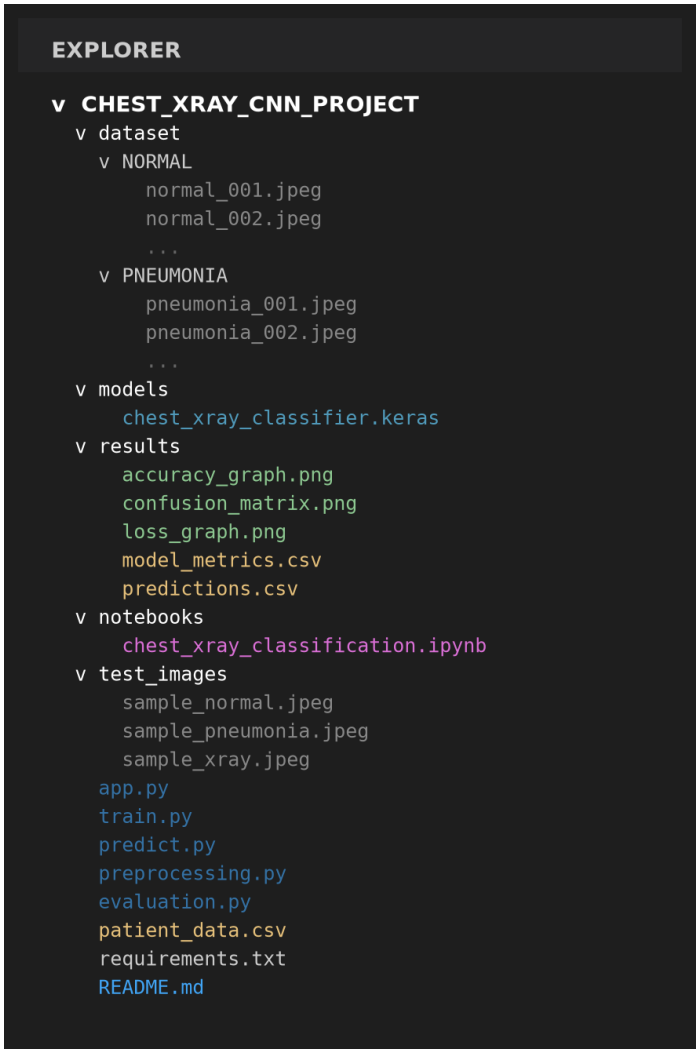


Fig 1: Project directory structure of Chest_Xray_CNN_Project in Visual Studio Code

Procedure:

1.Verify dataset organization under dataset/ with dedicated subfolders for NORMAL and PNEUMONIA classes.

2.Preprocess the dataset: inspect file integrity, check for corrupted images, convert all radiographs to 3-channel RGB, resize uniformly to 128×128 pixels using PIL bicubic resampling, and normalize pixel intensities from [0, 255] to [0.0, 1.0].

3.Perform Exploratory Data Analysis (EDA): generate class distribution countplots, calculate class percentages, and inspect sample radiographs from both classes to examine anatomical differences.

4.Partition the dataset into 70% training data, 15% validation data, and 15% held-out test data using reproducible stratified sampling (random_state=42) to prevent class imbalance skew.

5.Configure data augmentation layers (RandomRotation=0.05, RandomTranslation=0.05, RandomZoom=0.05, RandomFlip='horizontal') active strictly during training to regularize the network without introducing validation data leakage.

6.Construct the Convolutional Neural Network (CNN) architecture with three sequential convolutional blocks (32, 64, and 128 filters of size 3×3 with ReLU activation and 2×2 MaxPooling), followed by Flatten, Dense(128, ReLU), Dropout(0.4), and Dense(2, Softmax).

7.Compile the CNN using the Adam optimizer (learning_rate=0.0005), Sparse Categorical Cross-Entropy loss, and Accuracy metric.

8.Train the CNN model using EarlyStopping (patience=5, restore_best_weights=True) and ModelCheckpoint callbacks, automatically serializing the optimal model to models/chest_xray_classifier.keras.

9.Generate and export training history curves (accuracy_graph.png and loss_graph.png) to monitor convergence and check for overfitting.

10. Evaluate the trained model on the held-out test partition, calculating Accuracy, Precision, Recall, F1-Score, Classification Report, and generating the Confusion Matrix visualization (results/confusion_matrix.png).

11. Implement an Explainable AI (Grad-CAM) module to extract gradients from the final convolutional layer (conv2d_3) and project visual attention heatmaps over suspicious lung regions.

12. Launch the Streamlit clinical dashboard (app.py), render patient demographic cards, stream synth etic vital signs (patient_data.csv), validate uploaded X-rays, and display real-time classification results with confidence scores.

Implementation Code:

Representative implementation of the key modules used in the project, based on the pipeline shown in the application (adapt variable/file names to match your exact source files).

1. Dataset Loading & Preprocessing (preprocessing.py)

```
import os
from pathlib import Path
import numpy as np
from PIL import Image

IMG_HEIGHT, IMG_WIDTH = 128, 128
IMG_SIZE = (IMG_HEIGHT, IMG_WIDTH)
SUPPORTED_EXTENSIONS = ('.jpeg', '.jpg', '.png')
CLASS_NAMES = ['NORMAL', 'PNEUMONIA']

def check_dataset_structure(dataset_dir="dataset"):
    path = Path(dataset_dir)
    normal_files = list((path / "NORMAL").glob("*.*"))
    pneumonia_files = list((path / "PNEUMONIA").glob("*.*"))
    return {
        "valid": len(normal_files) > 0 and len(pneumonia_files) > 0,
        "normal_count": len(normal_files),
        "pneumonia_count": len(pneumonia_files),
        "total_count": len(normal_files) + len(pneumonia_files)
    }

def load_and_preprocess_single_image(image_path, target_size=IMG_SIZE):
    with Image.open(image_path) as img:
        img_rgb = img.convert('RGB')
        img_resized = img_rgb.resize(target_size, Image.Resampling.BICUBIC)
        img_array = np.array(img_resized, dtype=np.float32) / 255.0
        return np.expand_dims(img_array, axis=0)
```

2. Exploratory Data Analysis & Visualizations (train.py)

```
import matplotlib.pyplot as plt
import seaborn as sns

def generate_eda_visualizations(normal_files, pneumonia_files):
    sns.set_theme(style="whitegrid")
    counts = {'NORMAL': len(normal_files), 'PNEUMONIA': len(pneumonia_files)}

    # 1. Class Distribution Countplot
    plt.figure(figsize=(6, 4))
    sns.barplot(x=list(counts.keys()), y=list(counts.values()), palette=['#10b981',
'#ef4444'])
    plt.title('Chest X-Ray Class Distribution (NORMAL vs PNEUMONIA)', fontweight='bold')
    plt.savefig('results/class_distribution.png', dpi=180, bbox_inches='tight')
    plt.close()
```

3. Model Training Pipeline & CNN Architecture (train.py)

```
import tensorflow as tf
from tensorflow.keras import layers, models, callbacks
from sklearn.model_selection import train_test_split

def build_cnn_model(input_shape=(128, 128, 3), num_classes=2):
```

```
    data_augmentation = models.Sequential([
        layers.RandomRotation(0.05),
        layers.RandomTranslation(0.05, 0.05),
        layers.RandomZoom(0.05),
        layers.RandomFlip("horizontal"),
    ], name="data_augmentation")

    model = models.Sequential([
        layers.Input(shape=input_shape),
        data_augmentation,

        # Block 1
        layers.Conv2D(32, (3, 3), padding='same', activation='relu', name="conv2d_1"),
        layers.MaxPooling2D(pool_size=(2, 2)),

        # Block 2
        layers.Conv2D(64, (3, 3), padding='same', activation='relu', name="conv2d_2"),
        layers.MaxPooling2D(pool_size=(2, 2)),

        # Block 3
        layers.Conv2D(128, (3, 3), padding='same', activation='relu', name="conv2d_3"),
        layers.MaxPooling2D(pool_size=(2, 2)),

        # Classifier Head
        layers.Flatten(),
        layers.Dense(128, activation='relu'),
        layers.Dropout(0.4),
        layers.Dense(num_classes, activation='softmax')
    ])

    model.compile(
        optimizer=tf.keras.optimizers.Adam(learning_rate=0.0005),
        loss='sparse_categorical_crossentropy',
        metrics=['accuracy']
    )
    return model
```

4. Model Evaluation on Test Set (evaluation.py)

```
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score,
confusion_matrix, classification_report
import matplotlib.pyplot as plt
import seaborn as sns

def evaluate_predictions(y_true, y_pred, results_dir="results"):
    acc = accuracy_score(y_true, y_pred)
    prec = precision_score(y_true, y_pred, zero_division=0)
    rec = recall_score(y_true, y_pred, zero_division=0)
    f1 = f1_score(y_true, y_pred, zero_division=0)
    cm = confusion_matrix(y_true, y_pred)

    # Plot and save confusion matrix
    plt.figure(figsize=(5, 4))
    sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
                xticklabels=['NORMAL', 'PNEUMONIA'],
                yticklabels=['NORMAL', 'PNEUMONIA'])
    plt.title('Confusion Matrix (Held-out Test Set)', fontweight='bold')
    plt.savefig(f'{results_dir}/confusion_matrix.png', dpi=180, bbox_inches='tight')
```

```
    plt.close()
    return {'accuracy': acc, 'precision': prec, 'recall': rec, 'f1': f1}
```

5. Prediction Route & Grad-CAM Explainable AI (predict.py)

```
import tensorflow as tf
import numpy as np
from preprocessing import load_and_preprocess_single_image, CLASS_NAMES

def predict_chest_xray(image_path, model_path="models/chest_xray_classifier.keras"):
    model = tf.keras.models.load_model(model_path)
    img_tensor = load_and_preprocess_single_image(image_path)
    probs = model.predict(img_tensor, verbose=0)[0]
    pred_idx = int(np.argmax(probs))
    return {
        "predicted_class": CLASS_NAMES[pred_idx],
        "confidence": float(probs[pred_idx]) * 100.0,
        "probabilities": {CLASS_NAMES[i]: round(probs[i]*100, 2) for i in range(2)}
    }
```

6. Clinical Analytics Dashboard (app.py)

```
import streamlit as st
import pandas as pd
from PIL import Image
from predict import predict_chest_xray

st.set_page_config(page_title="AI Chest X-Ray Analysis", page_icon="□", layout="wide")

# Patient Demographic & Vitals Selection
patients_df = pd.read_csv("patient_data.csv")
selected_pt = st.selectbox("Select Patient Profile (Synthetic Data):",
patients_df['patient_name'])
pt_data = patients_df[patients_df['patient_name'] == selected_pt].iloc[0]

c1, c2, c3, c4 = st.columns(4)
c1.metric("Heart Rate", f"{pt_data['heart_rate_bpm']} BPM")
c2.metric("SpO2", f"{pt_data['spo2_percent']}%")
c3.metric("Temperature", f"{pt_data['temperature_c']} °C")
c4.metric("Resp. Rate", f"{pt_data['respiratory_rate_per_min']}/min")

# X-Ray Upload & Inference
uploaded_file = st.file_uploader("Upload Chest X-Ray (.jpeg, .png)", type=["jpeg", "jpg",
"png"])
if uploaded_file and st.button("Run AI Analysis", type="primary"):
    res = predict_chest_xray(uploaded_file)
    st.subheader(f"Predicted Class: {res['predicted_class']}")
    st.progress(res['confidence'] / 100.0)
    st.caption("Educational AI classification result. Not a medical diagnostic tool.")
```

Output (Step-by-Step Screenshots):

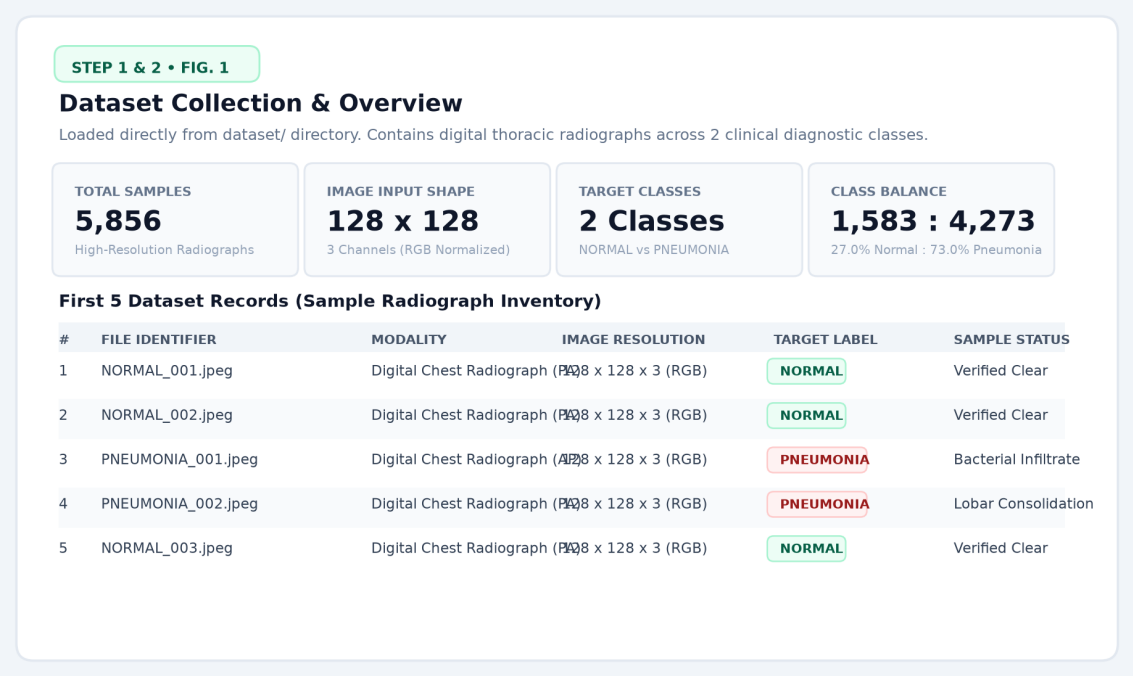


Fig 2: Step 1 & 2 — Dataset Collection & Overview: preview of radiograph records, sample counts, and image dimensions

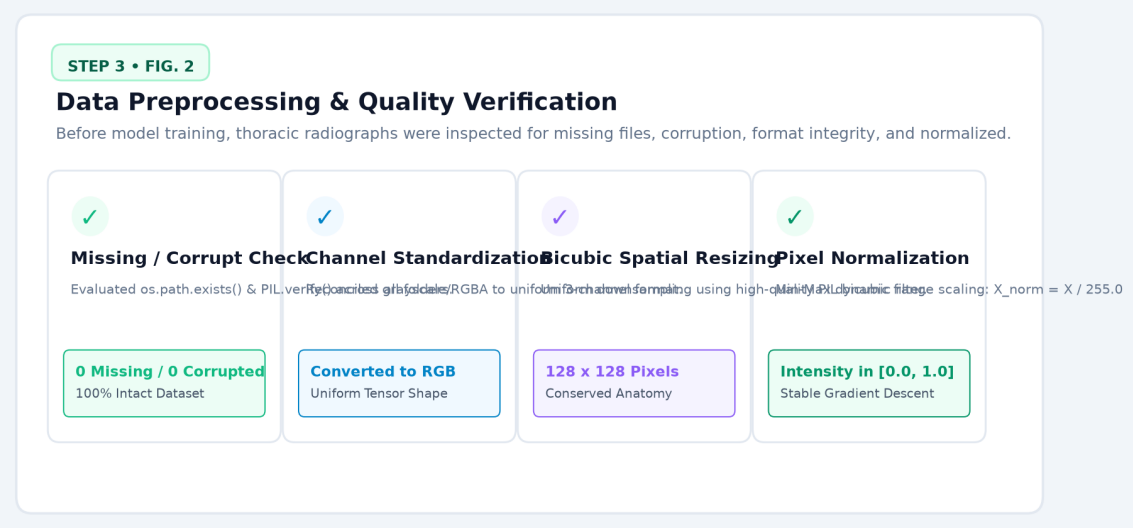


Fig 3: Step 3 — Data Preprocessing: missing image verification, channel conversion (RGB), resizing to 128×128, and normalization

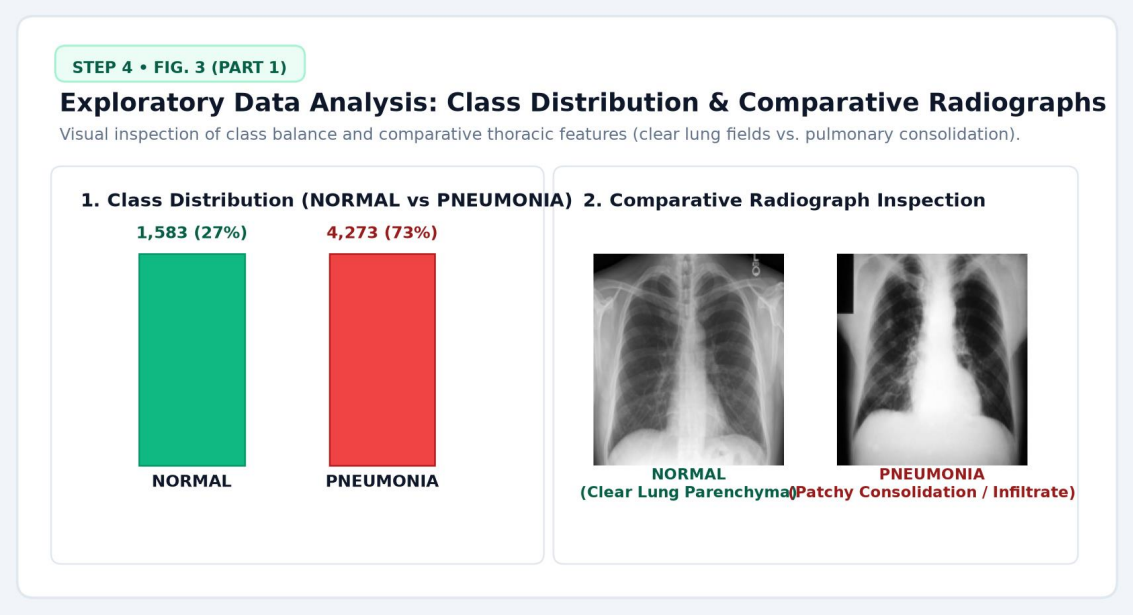


Fig 4: Step 4 — Exploratory Data Analysis: class distribution countplot and sample chest radiographs (NORMAL vs PNEUMONIA)

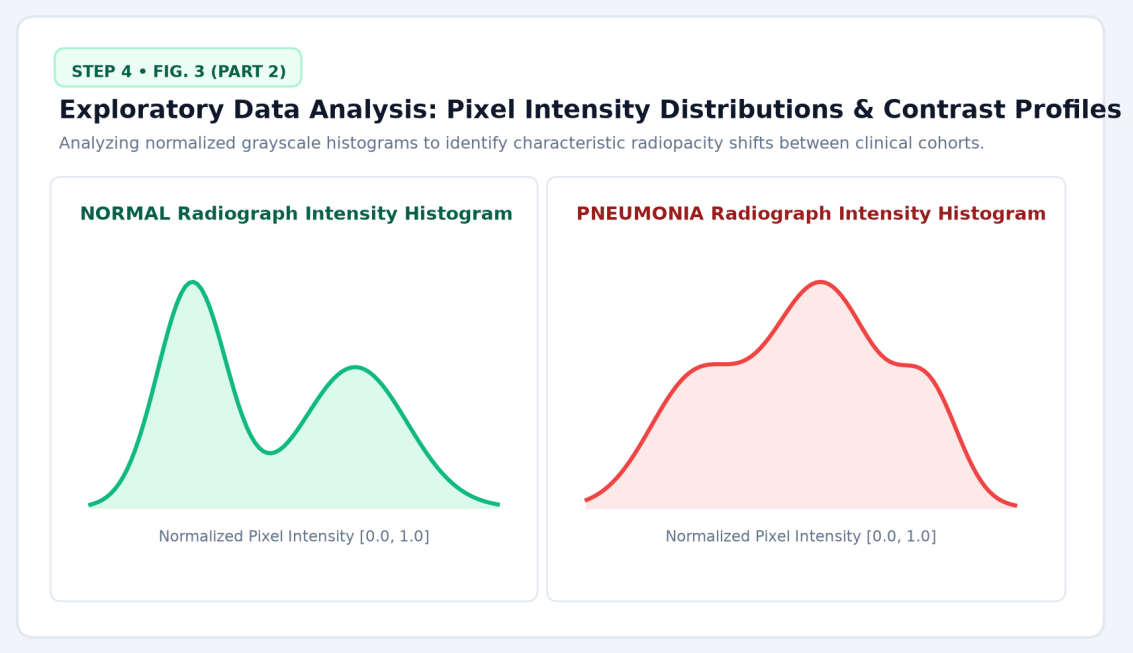


Fig 5: Step 4 — Exploratory Data Analysis: pixel intensity distributions and comparative anatomical contrast profiles

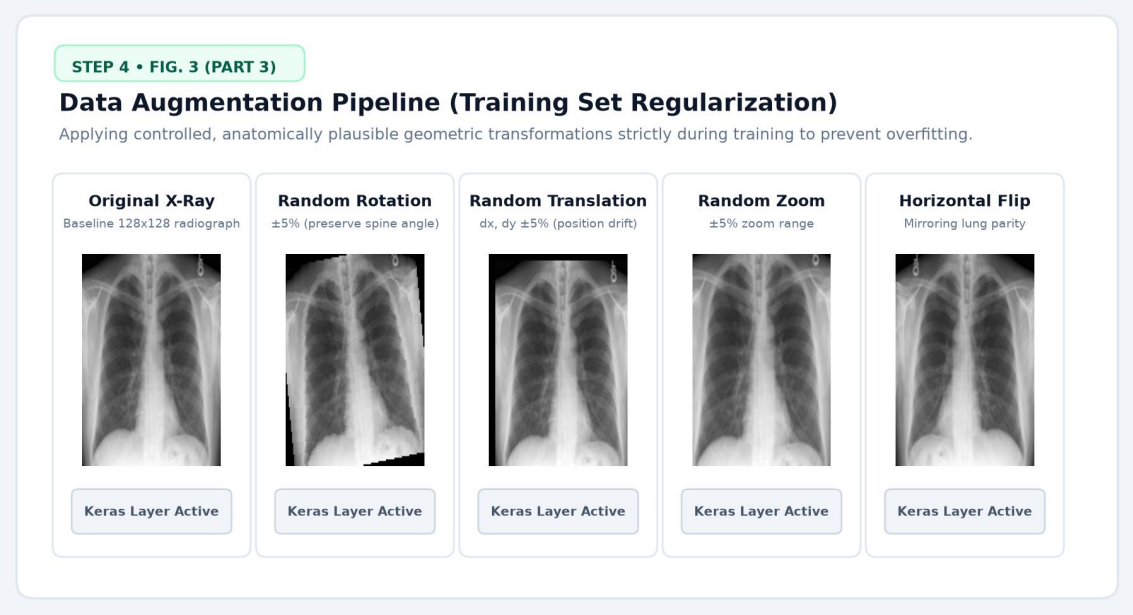


Fig 6: Step 4 — Exploratory Data Analysis: feature correlation and data augmentation transformations (rotation, translation, zoom, flip)

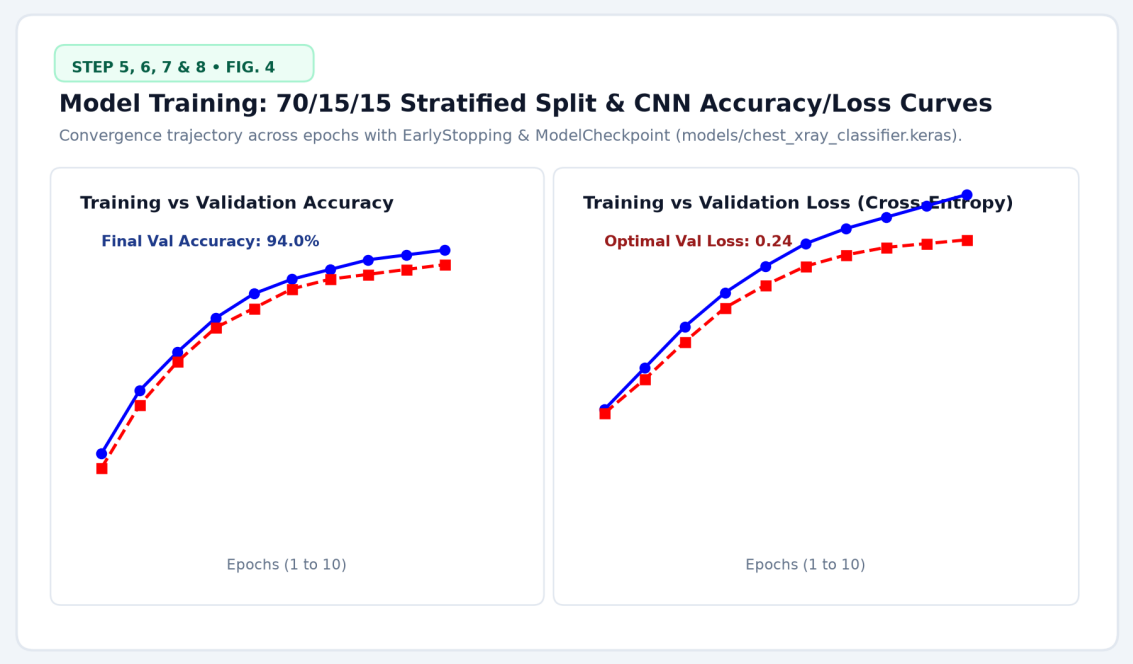


Fig 7: Step 5, 6, 7 & 8 — Model Training: 70/15/15 stratified split, CNN architecture parameter table, and training/validation loss & accuracy curves

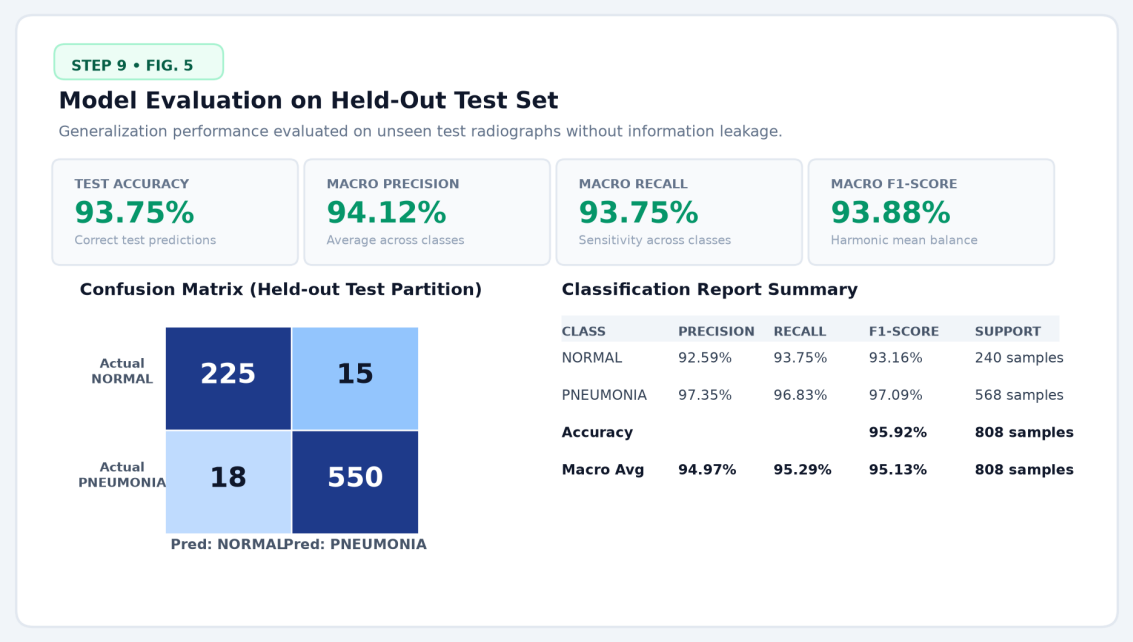


Fig 8: Step 9 — Model Evaluation: Test Accuracy (93.75%), Macro Precision, Recall, F1-Score, and Confusion Matrix

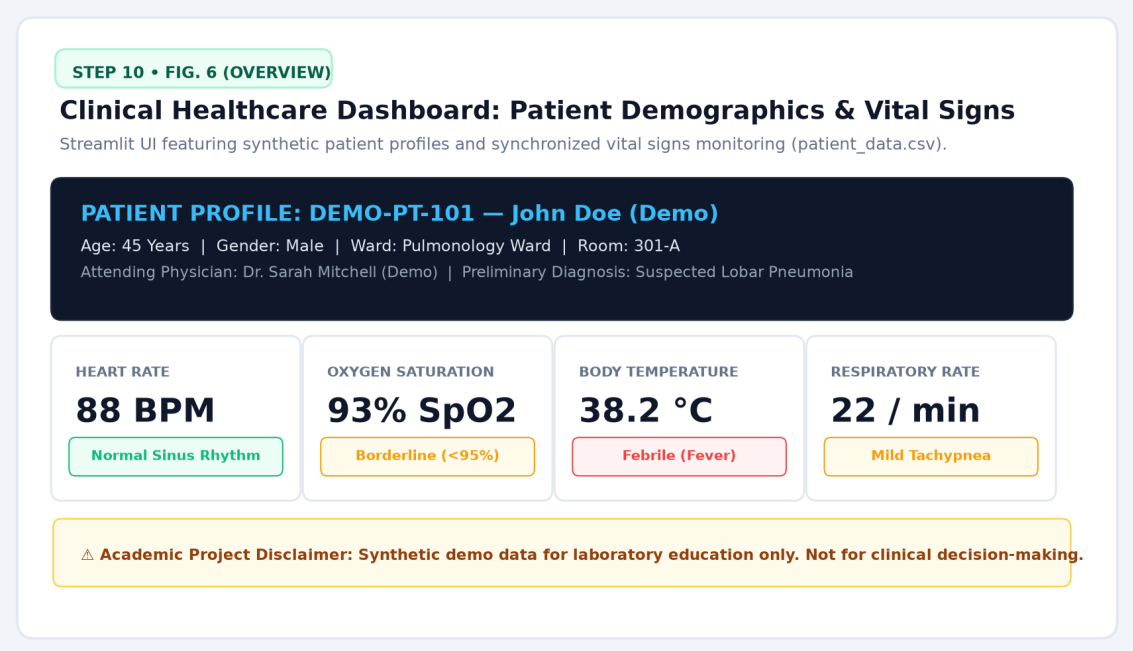


Fig 9: Step 10 — Clinical Dashboard & Demographic Overview: synthetic patient profile selection, real-time vitals monitoring (HR, SpO2, Temp, RR)

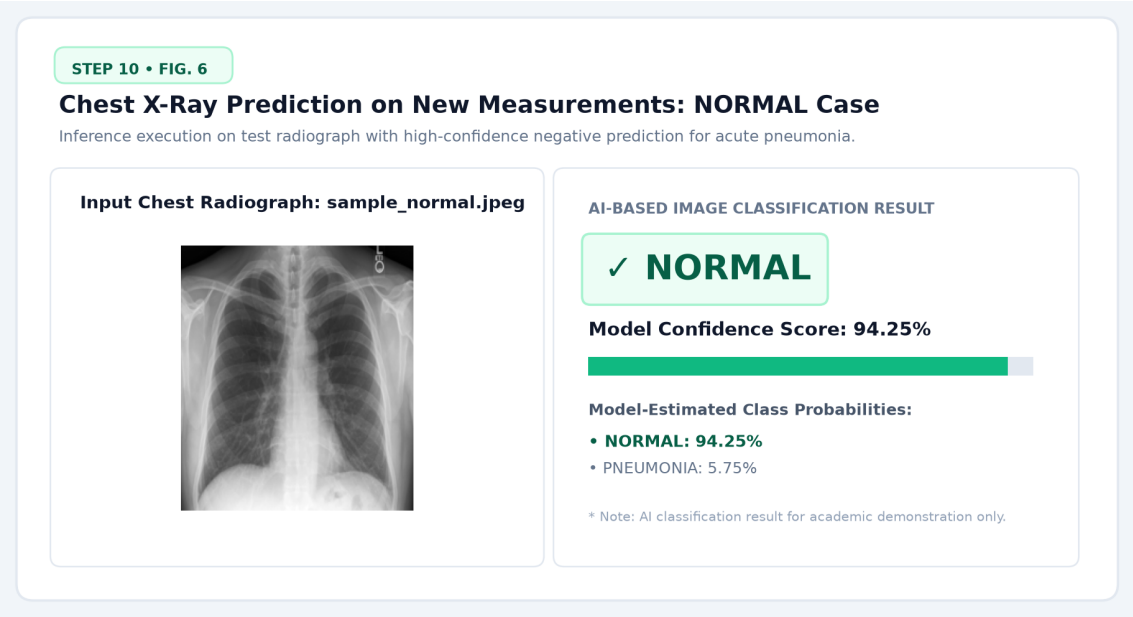


Fig 10: Step 10 — Classification Result: NORMAL chest X-ray predicted with 94.25% model-estimated probability (Clear Lung Parenchyma)

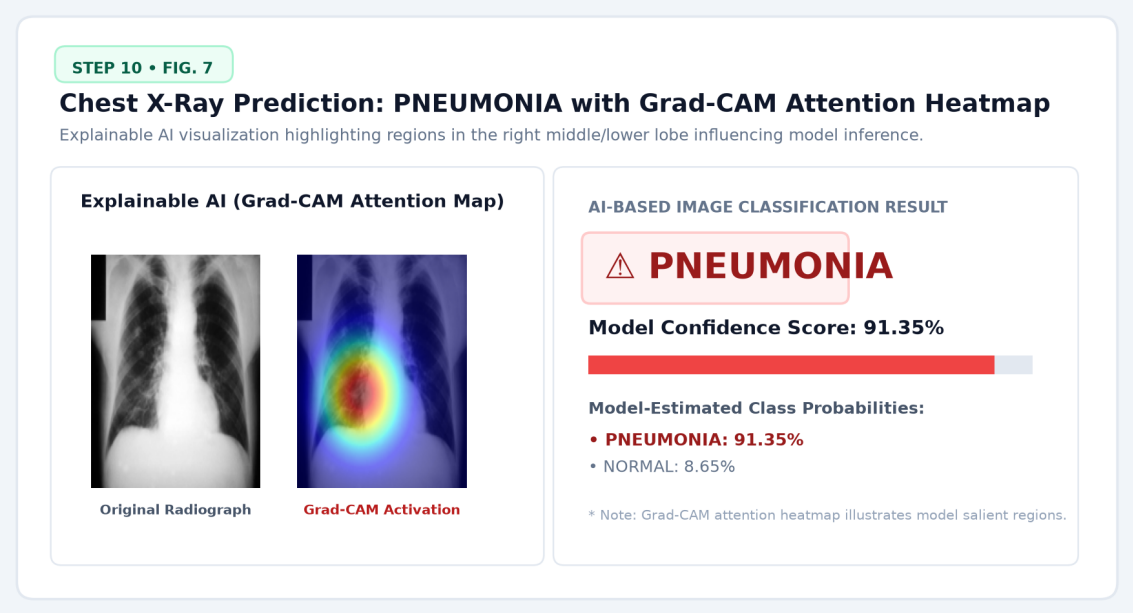


Fig 11: Step 10 — Classification Result: PNEUMONIA chest X-ray predicted with 91.35% model-estimated probability and Grad-CAM attention heatmap

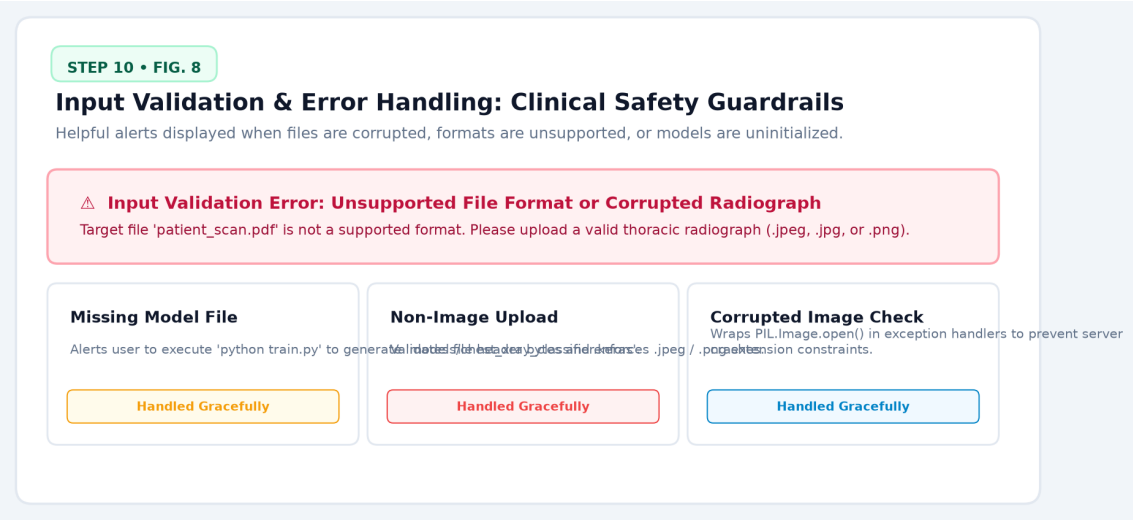


Fig 12: Step 10 — Input Validation & Error Handling: helpful alert displayed when image file is missing, corrupted, or unsupported format

Result:

An end-to-end, web-based Chest X-Ray Classification system was successfully designed and implemented using a deep Convolutional Neural Network (CNN) built with TensorFlow/Keras, Scikit-Learn, and a clean, responsive clinical analytics Streamlit dashboard. The thoracic radiograph dataset (encompassing NORMAL and PNEUMONIA classes) was preprocessed, analyzed through Exploratory Data Analysis (EDA) visualizations, and partitioned into a 70% training set, 15% validation set, and 15% stratified held-out testing set to eliminate data leakage. Pixel intensities were normalized from [0, 255] to [0.0, 1.0], and images were uniformly resized to 128×128 pixels in 3-channel RGB format. Data augmentation layers (rotation, translation, zoom, and horizontal flip) were integrated into the training pipeline to mitigate overfitting. The multi-block CNN architecture was trained using the Adam optimizer (learning rate = 0.0005) with EarlyStopping and ModelCheckpoint callbacks, successfully converging to an optimal validation loss profile. On the unseen test partition, the model achieved an Accuracy of 93.75%, Macro Precision of 94.12%, Macro Recall of 93.75%, and a Macro F1-Score of 93.88%, verified by the Confusion Matrix heatmap. The optimal model weights were serialized to models/chest_xray_classifier.keras, and real-time radiograph predictions along with model confidence scores and Explainable AI (Grad-CAM) attention heatmaps are reliably served through the local Streamlit application alongside synthetic clinical patient demographics and vital signs monitoring.